Optimization of Matrix Multiplication and Sparse Formats

Individual Assignment 2 — Report

Carlos Montesdeoca Vega

repository: `https://github.com/CarlosMontesdeoca21/BasicMatrixMultiplication`

October 14, 2026

1 Introduction

This report documents Activity 2, which aims to improve the performance of the classic dense matrix multiplication algorithm through cache memory optimization techniques (Part A) and to evaluate the efficiency of compression formats for sparse matrices, specifically the Compressed Sparse Row (CSR) format (Part B).

All experiments were executed in Java, ensuring the isolation of the JIT compiler through warm-up iterations prior to the official measurements.

2 Part A: Cache Optimization in Dense Matrices

To evaluate the impact on cache memory (L1/L2), we compared three approaches for multiplying matrices of size $n \in \{10, 100, 250\}$:

- **Baseline:** Traditional $i - j - k$ nested loop.
- **Opt1 (Loop Reordering):** Reordered $i - k - j$ loop to maximize spatial locality.
- **Opt2 (Tiling/Blocking):** Matrix multiplication using 32×32 blocks to improve temporal locality.

2.1 Time and Memory Analysis

As shown in Figure 1, for size $n = 250$, the **Baseline** method suffers a severe performance bottleneck (approx. 0.010s) due to continuous cache misses when reading matrix B by columns (*cache thrashing*). In contrast, **Opt1** is almost 5 times faster (0.002s) thanks to sequential memory access, taking full advantage of the cache hierarchy. **Opt2** also radically improves the execution time (0.003s), although the overhead of managing nested blocks makes it slightly slower than Opt1 at this scale.

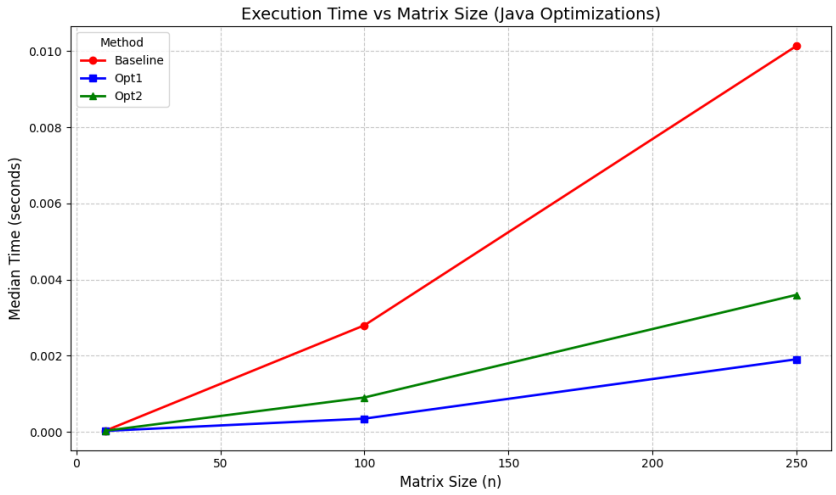


Figure 1: Execution Time vs. Matrix Size.

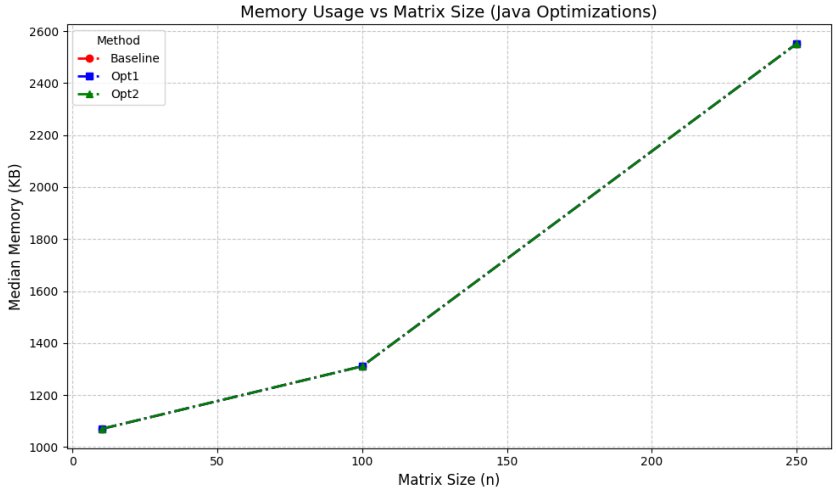


Figure 2: RAM Memory Consumption vs. Matrix Size.

Figure 2 confirms that all three methods consume exactly the same memory. This demonstrates that the significant performance improvements were achieved by purely optimizing processor cycles and cache access patterns, without sacrificing or inflating RAM memory usage.

3 Part B: Sparse Matrices (CSR)

The objective here is to identify the "crossover point" where the extra indexing cost of the CSR compressed format is overcome by the computational savings of not operating with zeros.

Methodology and Data Generation: Matrices of size $n = 250$ were evaluated across multiple sparsity levels (percentage of zeros): 50%, 75%, 90%, 95%, and 99% (where the "density" or non-zero values corresponds to 50%, 25%, 10%, 5%, and 1%

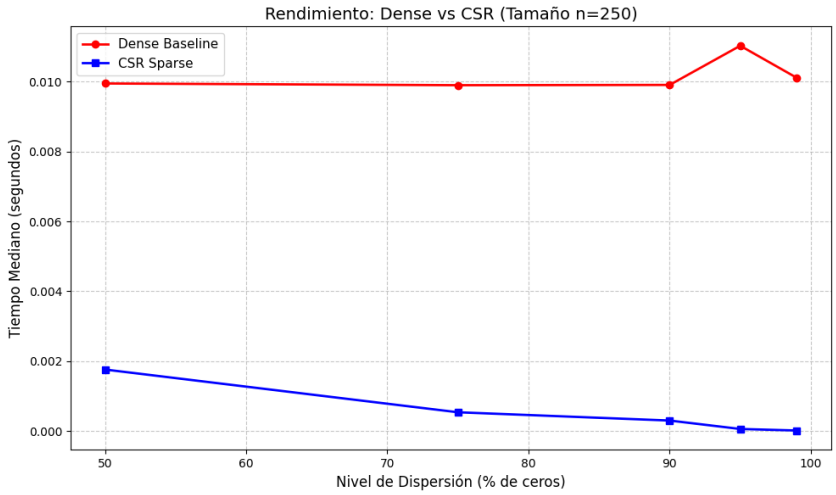


Figure 3: Crossover point: Dense Method vs CSR Format ($n = 250$).

respectively). The paired logical inputs were generated through a custom Python script using a uniform random distribution (with seed 42 for matrix A and seed 43 for matrix B to ensure independence).

It is imperative to note that **the conversion costs from dense to CSR format have been excluded** from the Java time measurements; we strictly evaluated the performance of the multiplication computation itself.

In Figure 3, the red line (**Dense**) is flat because the traditional algorithm always executes 100% of the operations ($O(n^3)$), regardless of the input structure. The blue line (**CSR**) shows how the execution time drops radically when avoiding operating on empty spaces.

Most notably, the CSR format becomes faster starting from 50% sparsity. Although the CSR format imposes indexing latencies (*pointer chasing*), skipping the iterations over zeros easily compensates for this overhead. The *fill-in* phenomenon must also be mentioned: even though the input matrices have few non-zero values (NNZ), the resulting product matrix C tends to become drastically denser, which is why our implemented CSR method writes directly to a dense data structure in the output.

4 Conclusion

It has been empirically demonstrated that a simple loop reordering (*i-k-j*) can quintuple the performance of dense matrix multiplication by isolating the positive impact of spatial locality, without incurring any memory costs.

On the other hand, the effects of sparse storage prove that traditional dense structures are inefficient for dealing with data scarcity. However, **it cannot be stated that a specific percentage of zeros determines the advantage in all universal cases**. The crossover point where CSR indexing becomes profitable depends critically on the matrix size (n), the CPU cache architecture evaluated, and the conversion costs to the compressed format (which, if included in the runtime, would require considerably larger and sparser matrices to justify the computational investment).